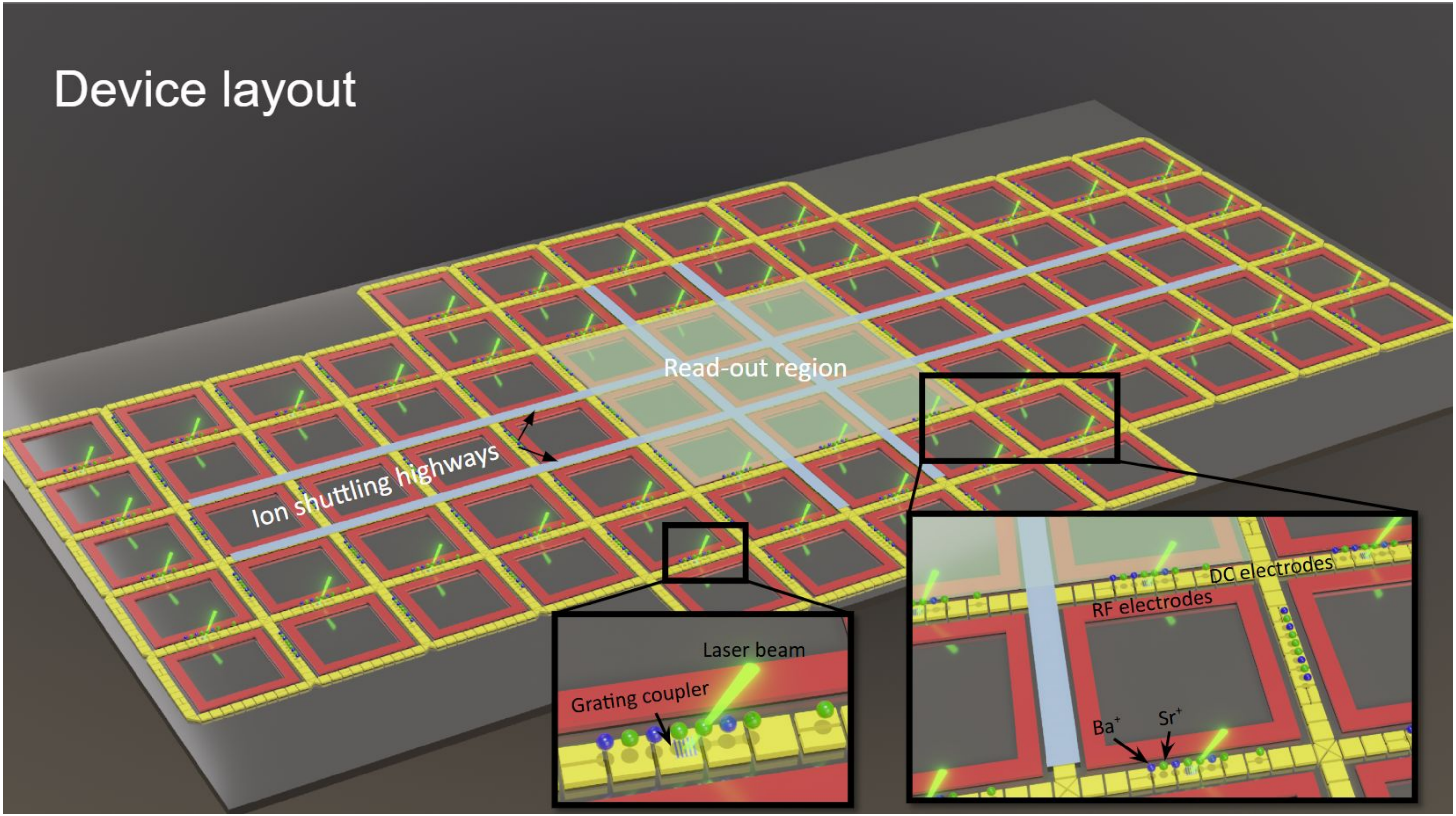
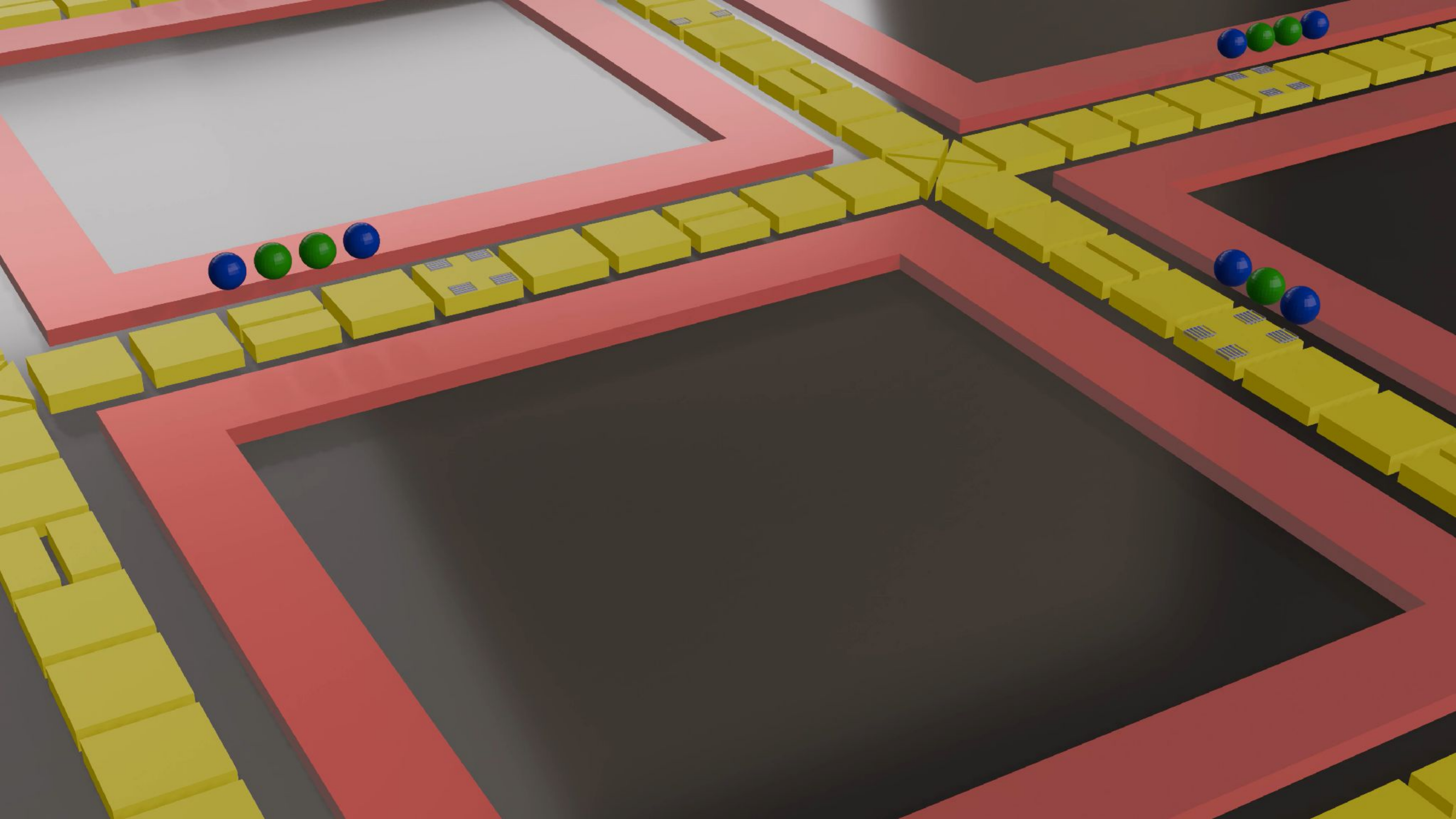
Architecture of FTQC using trapped ions - QCCD

Ke Sun
Postdoc @ UC Berkeley

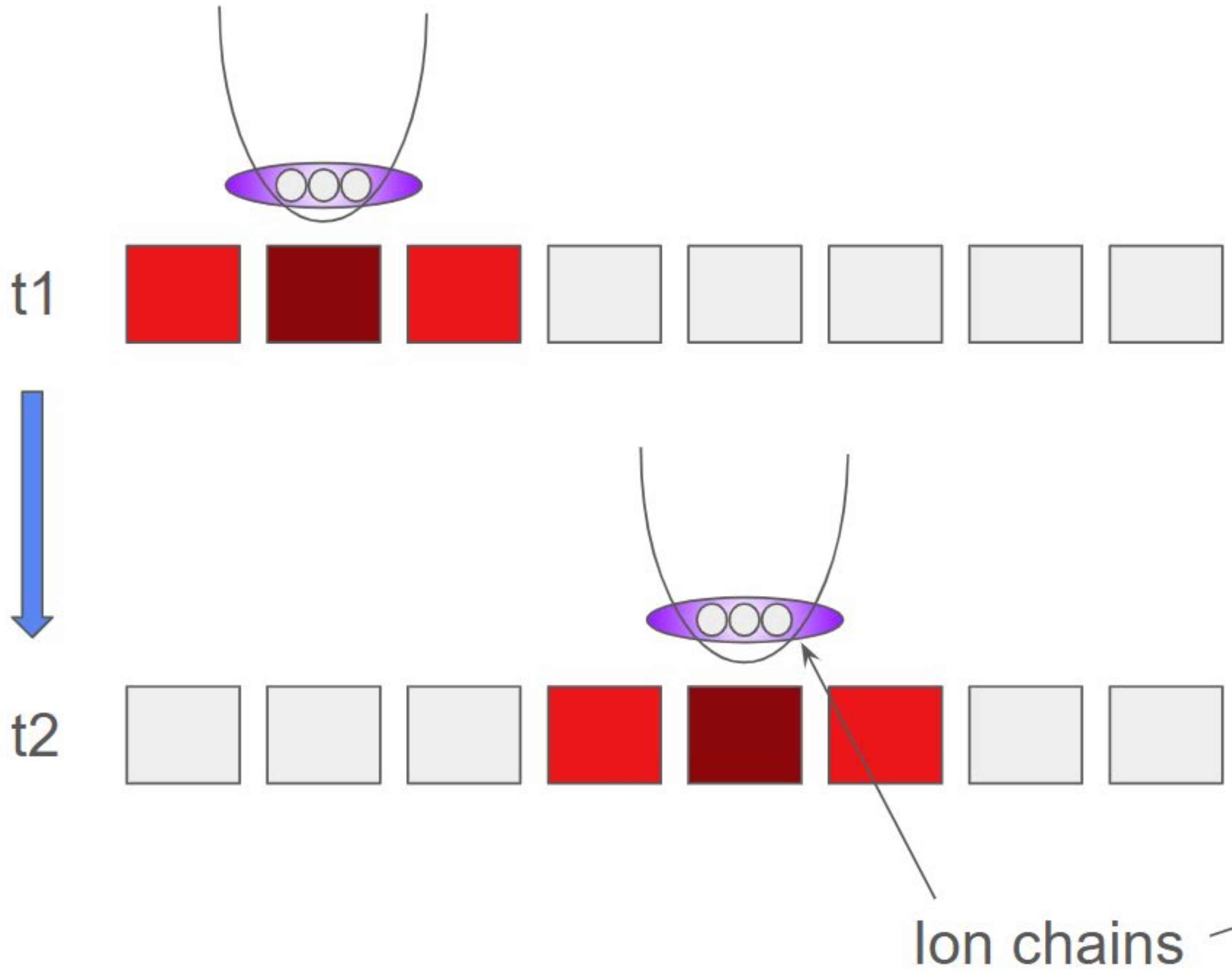
QCCD layout



Device layout
Read-out region
Ion shuttling highways
Laser beam
Grating coupler
DC electrodes
RF electrodes
Ba+
Sr+

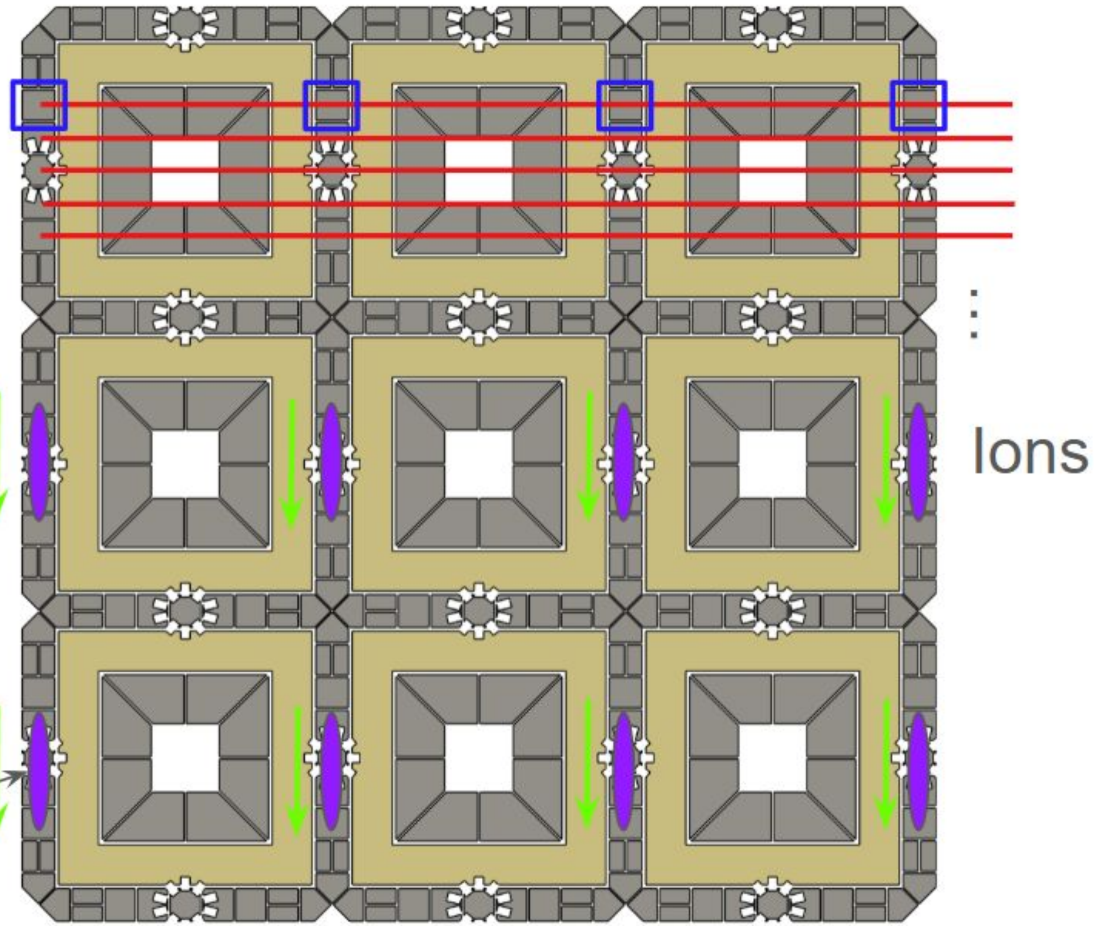


Ion shuttling



- Broadcasting:

  DC electrodes in different zones are controlled by the same channel.

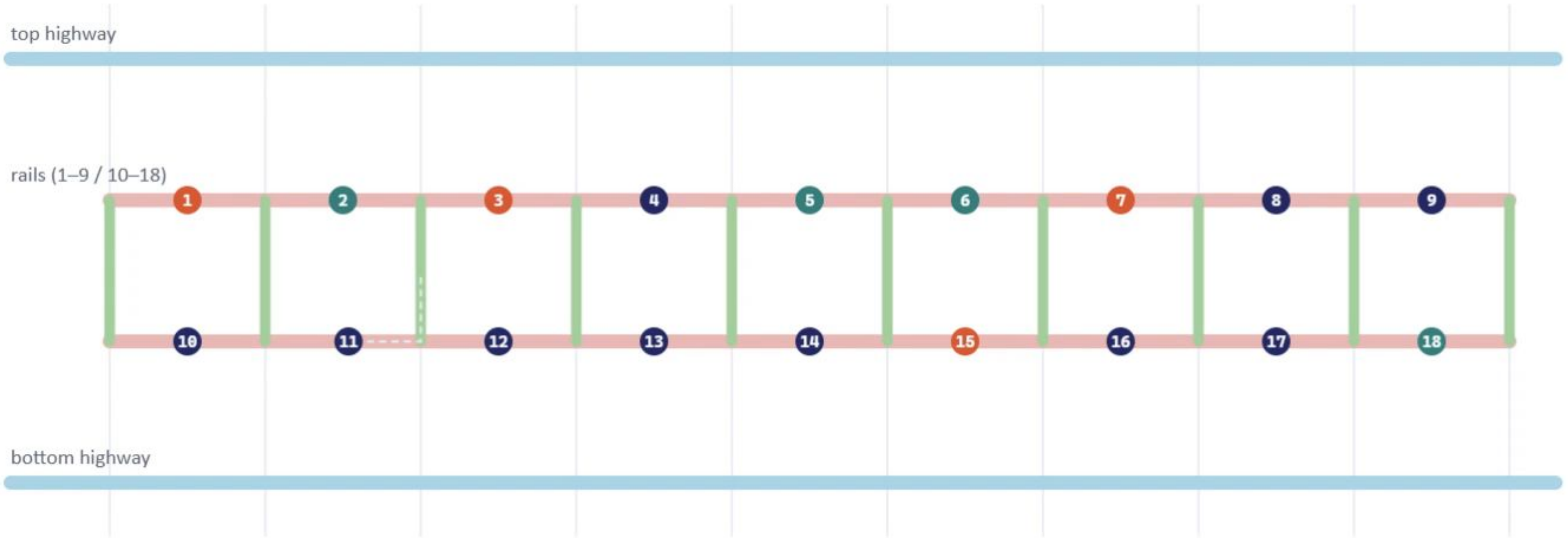


SCHEME B

Highway Shuttling — eject · shuttle · re-insert

STEPS 0/46 COST 0/52

Stabilizers (1,3,7,15) and (2,5,6,18). Schedule produced by routing_planner_B.py: spectators are routed clear first (11, 13, 4 up; 10 to the left highway overhang; 12 right), then movers thread the vacated segments — every vertical transfer is two 90° junction moves along real edges, ion 2 dodges onto a boundary vertical so 3 can slide past, and only identical hops share a step.



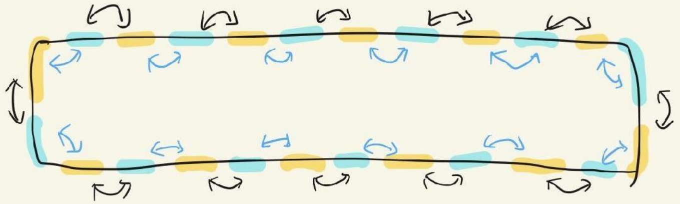
Step 1 — ion 11 · 90° turn right→up (1 step, cost 1) `[steps 1 · cost 1]`

Pause | Next phase | Reset | 1×

(1,3,7,15) (2,5,6,18) spectator data region highway computing region

IonQ

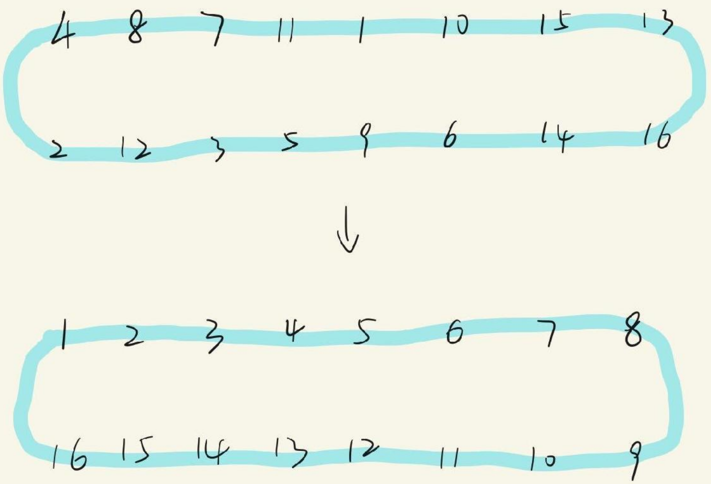
WISE:



2 Basic operations

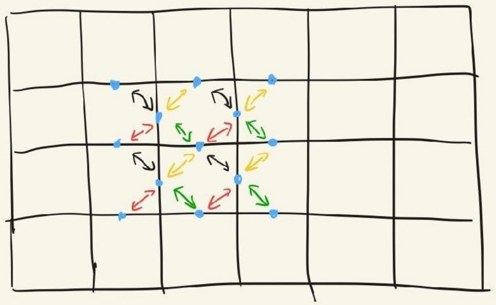
① Same color can be implemented simultaneously.

② Can determine swap or not.



Quantinuum

C2LR:



4 Basic operations

① Can be implemented simultaneously.

② Can determine whether they experience the swap or not.

Equivalent to re-arrange

3 7 13		1 2 3
6 5		4 5
1 9 11	⇒	6 7 8
12 4		9 10
8 2 10		11 12 13.

Multi-Pass Greedy Ion Routing

QCCD ion-routing algorithm — Jones & Murali, "Architecting Scalable Trapped Ion Quantum Computers using Surface Codes," ASPLOS '26 (arXiv:2510.23519)

Priority queue (this pass): A1 > A2 > A3 > A4 *(routed this pass)* *(blocked — wait)*

Hardware Constraints

- **Trap capacity:** fixed max ion count per trap
- **Junction exclusivity:** one ion per junction
- **Segment exclusivity:** one ion per shuttling segment

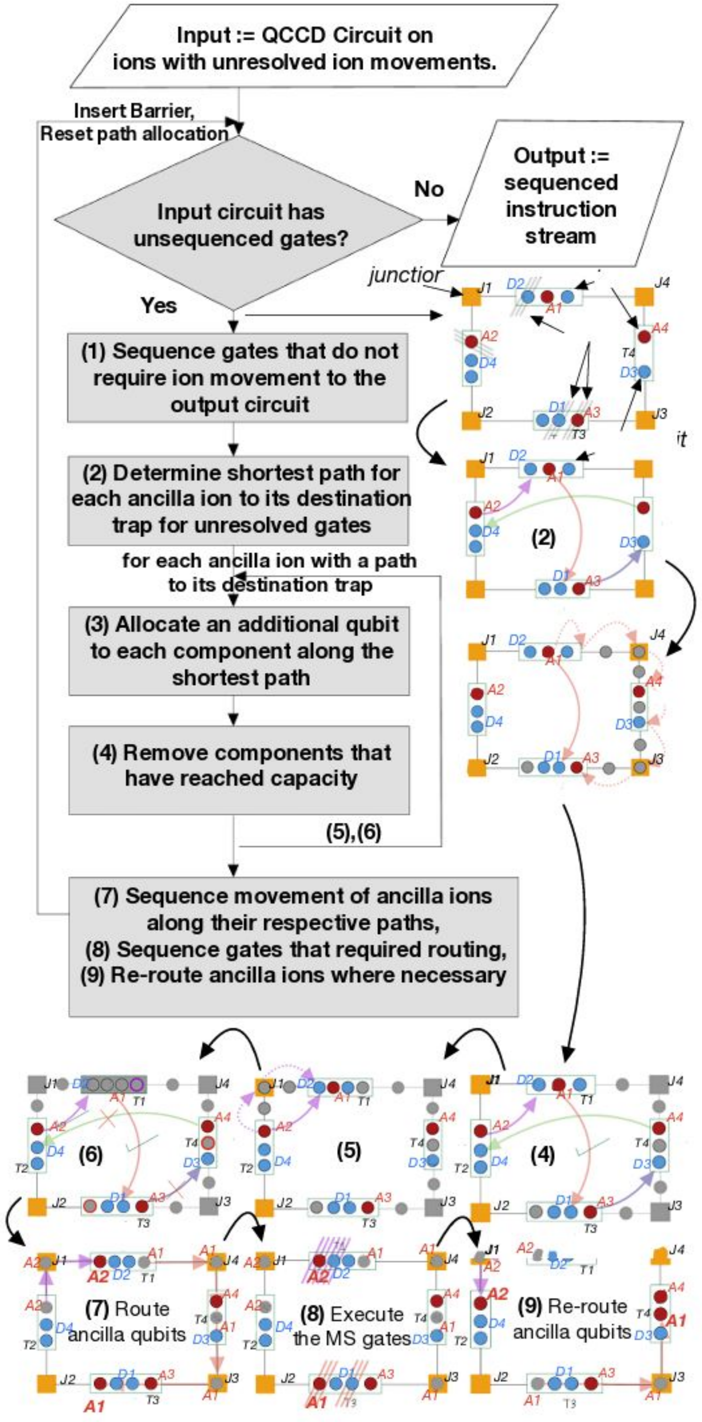
Invariant: at pass start/end, every trap sits ≥ 1 below capacity, and no junction or segment holds an ion.

3.85×

less ion-movement time vs. QCCDSim / Muzzle the Shuttle

1.04×

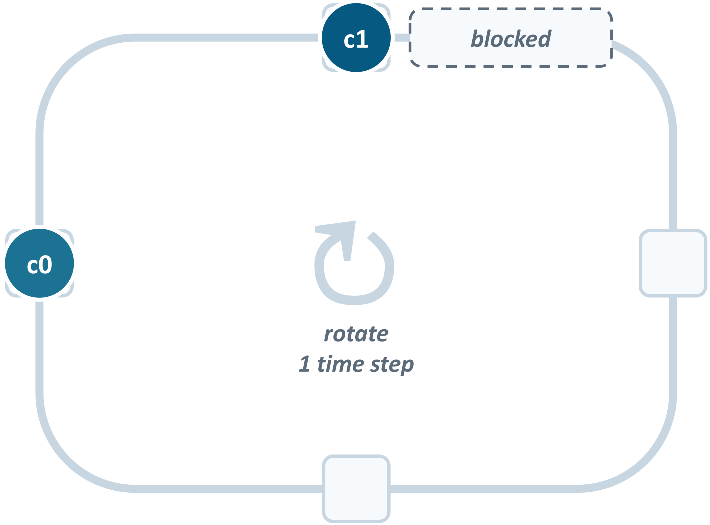
of the theoretical-minimum routing operation count



Cycle-Based Shuttling

Conflict-free ion routing via graph cycles — Schoenberger, Hillmich, Brandl & Wille, “Shuttling for Scalable Trapped-Ion Quantum Computers,” IEEE TCAD 2024 (arXiv:2402.14065)

c0 wants to move north — slot already occupied by **c1**



After rotation: c0 moves W⟶N (advances) • c1 moves N⟶E (displaced, not blocked)

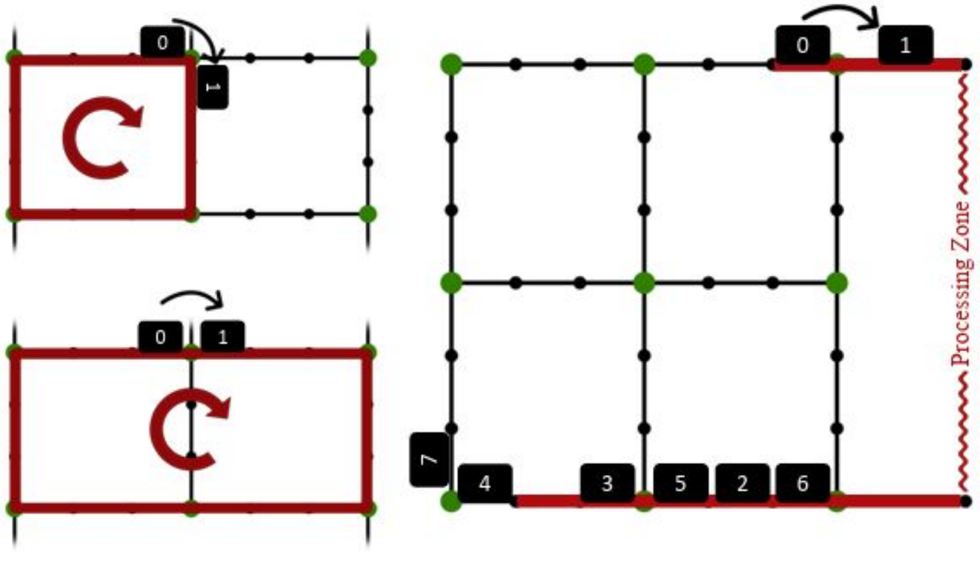
Vertical move
⟶ 1 rectangle

Horizontal move
⟶ 2 rectangles

Conflicting cycles: if two cycles overlap and rotate in opposite directions, only the cycle carrying the higher-priority chain executes this step.

How Cycle Rotation Works

- **Graph abstraction:**
  junctions are nodes, chain positions are edges
- **Detect a block:**
  shortest path of a chain is occupied by another
- **Build a cycle:**
  form a closed loop along that shortest path
- **Rotate together:**
  every chain on the cycle shifts one edge, same direction



(a) In memory zone (b) Through processing zone

A ring beats a grid for non-topological QEC

High-rate qLDPC codes (hypergraph product, bivariate bicycle) need long-range stabilizer connections that standard 2D-grid trapped-ion architectures cannot route without collisions.

THE PROBLEM

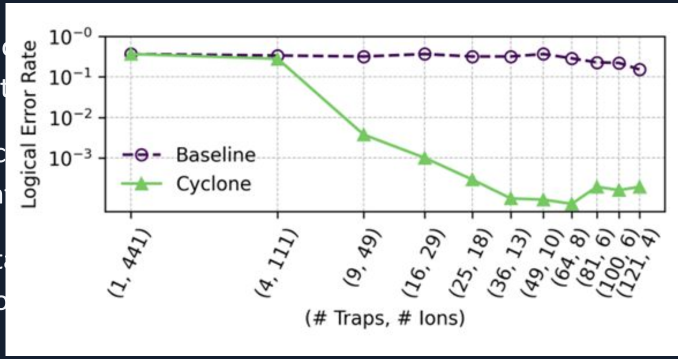
- **2D grids serialize non-local codes**
  Busy shuttling paths block one another ("roadblocks") whenever a stabilizer's data qubits sit far apart on the grid.

- **Roadblocks destroy inherent parallelism**
  Circuits that are highly parallelizable on paper get fully serialized in practice — rendering some codes unusable on the baseline grid.

- **Neither fix alone works**
  Better hardware with the wrong schedule, or a smarter schedule on the wrong hardware, both underperform the baseline (Fig. 6). It takes a codesign.

THE FIX — CYCLONE

↻ *all ancilla move together, one direction, every step*



- Ring top… rners, the cheapest…
- Strict loc… ce, so no path is ever con…
- 2 full rot… d 2 reuses the ancilla fo…

Wins on every axis — and the gain grows with code size

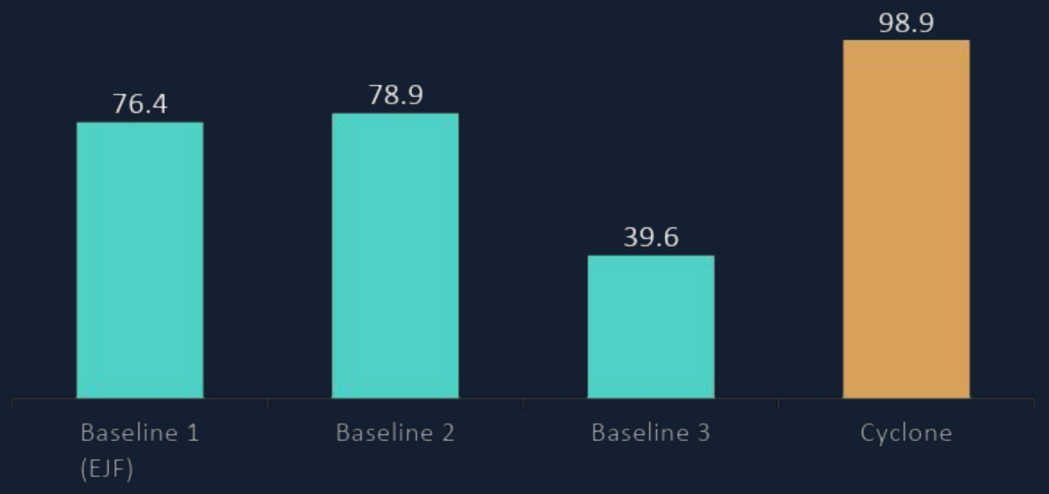
4×
execution-time speedup vs. baseline grid

2–3 ords.
of magnitude lower logical error rate

~20×
better spacetime efficiency (traps × time × ancilla)

O(1)
DACs required — vs. O(traps) for a grid

REALIZED PARALLELIZATION BY COMPILER (Fig. 20)



BASELINE GRID vs. CYCLONE

	Baseline grid	Cyclone
Traps needed	O(n), dense mesh	m/2 (~half the traps)
Junction degree	up to 4 (120 µs cross)	2 only (10 µs cross)
DAC scaling	one per trap	constant, broadcast
Roadblocks	present, worsen with scale	none, by construction
BB code [[144,12,12]]	only code that error-corrects	all tested codes error-correct

Sensitivity checks rule out confounds: neither looser baseline trap capacity, nor two alternative baseline compilers, nor a lower-cost swap primitive close the gap — the advantage is architectural.

Architecture Decision Path

Two questions determine the routing architecture — first the transport style, then the zone occupancy.

Q1 · All free movement, or cyclone?

Free movement ✗

More roadblocks — ions obstruct each other's paths and transport serializes.

Cyclone (rotation) ✓ chosen

More parallel operations — synchronized cyclic transport batches identical moves.

Q2 · Fixed number of ions in each zone? Yes — two ways to do it:

① Long ion chains not ideal ✗

Individual optical addressing is required (PIC).
WISE can move idle zones away — but cannot decide which ion in which zone is addressed.
→ a switch on every beam would be needed.

Open question: metasurface + DMD might make this workable.

② Small ion chains (≤ 2) ✓ chosen

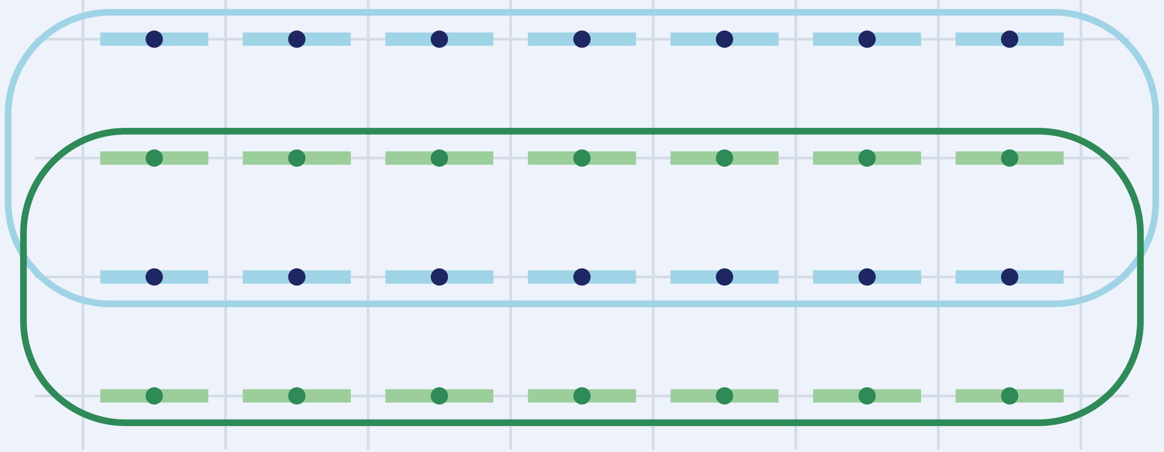
A global beam suffices — no individual addressing.
Price: more laser power, because of the large beam size.
Every trapping zone holds only 1–2 data qubits; ions are moved away to avoid unwanted interactions.
In the end each zone holds exactly one ion — otherwise individual addressing returns.

⇒ **WISE architecture + cyclone:** one ion per zone, globally addressed, rotation-based transport.

Cyclone — but 1 ion per trap

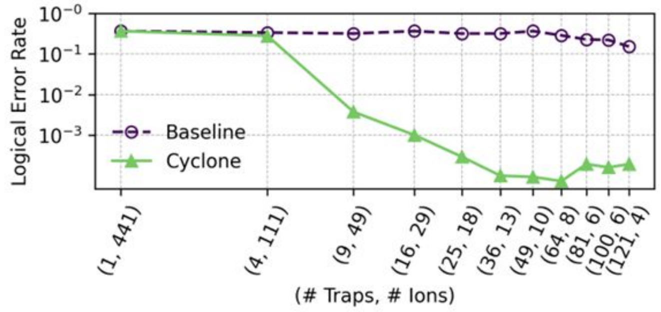
The data loop stays still while the ancilla loop rotates past it; the candidates then compare on time and space.

Data loop — stays still · Ancilla loop — rotates ⟳



1 round finishes one syndrome type ⟶ 2 rounds complete the full ESM. Space: Data : Ancilla = 1 : 1.

- 1 data loop + 1 ancilla loop
- 1 loop is enough to finish 1 round of X (or Z) stabilizer measurement.
- But 1 ion per trap is not scalable – **not space efficient**

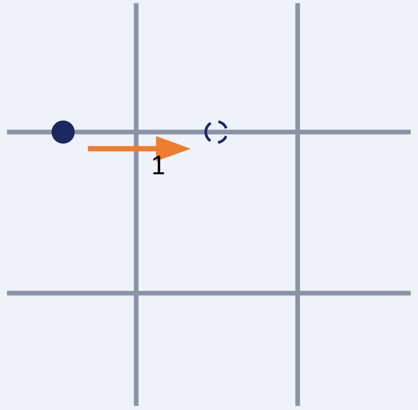


TRANSPORT PRIMITIVE 1

Single-Ion Move

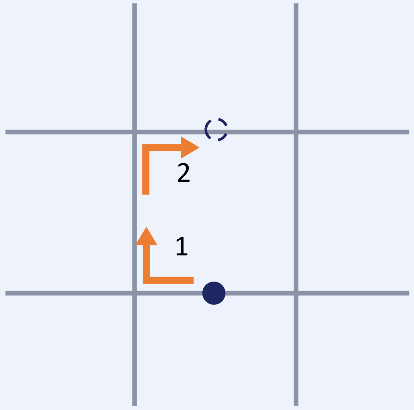
Moving one ion between segments — straight shuttling is cheap; corners and junction crossings cost one extra step.

1 Same segment, adjacent



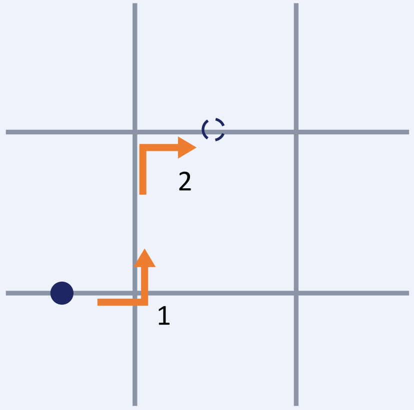
1 steps · **cost 1**

2 Across parallel rails



2 steps · **cost 2**

3 Diagonal across junction



2 steps · **cost 2**

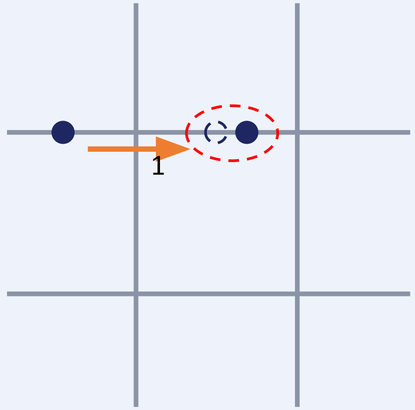
Cost rule (shuttle-induced error): 1 per segment-length shuttle; passing or turning at an X-junction adds one step. Solid = start, dashed = destination.
Step rule (time cost): same types of basic operations can be executed in the same step.

TRANSPORT PRIMITIVE 2

Move + Merge

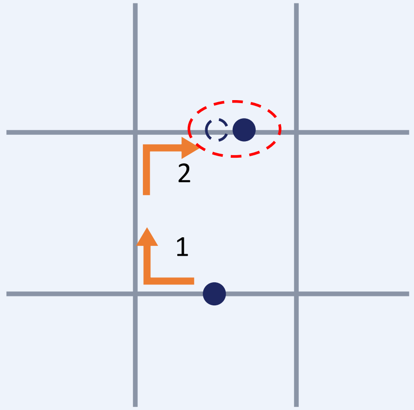
Shuttling one ion into another segment and merging into a shared potential well — plus the DC electrode budget it implies.

1 Same segment, adjacent



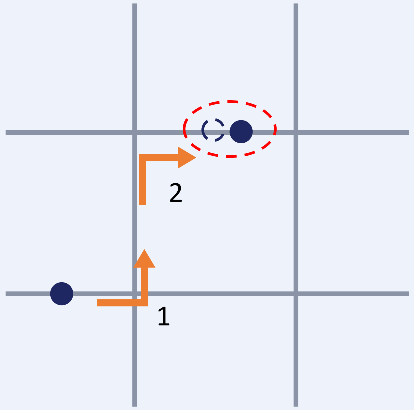
1 steps · cost 1

2 Across parallel rails



2 steps · cost 2

3 Diagonal across junction



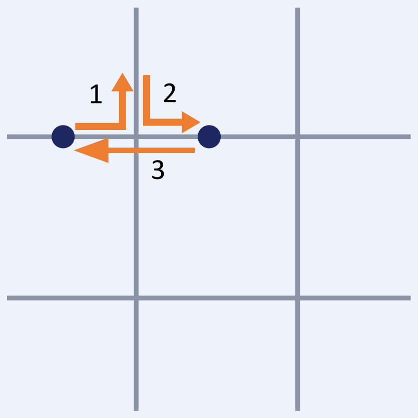
2 steps · cost 2

TRANSPORT PRIMITIVE 3

Ion Swap

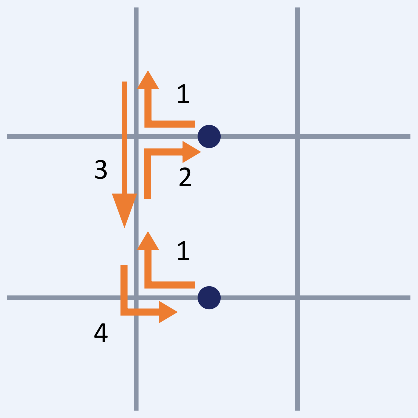
Exchanging two ions through segments and X-junctions of the trap grid — cost depends on the relative configuration.

1 Same segment, adjacent



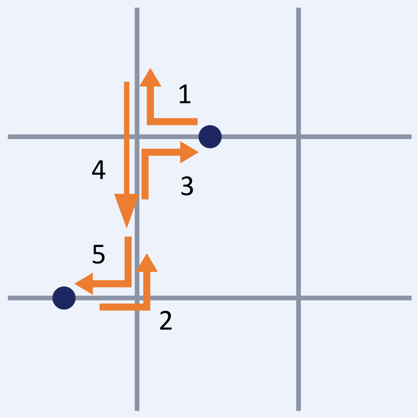
3 steps · **cost 3**

2 Across parallel rails



4 steps · **cost 5**

3 Diagonal across junction



5 steps · **cost 5**

Odd–even transposition sorting: each sorting round applies swaps in two batches (×2).

ROUTING SCHEME A

Ring Rotation — Animated

16-ion ring; only identical hops (same in/out junction direction) share a time step. Corner ions 7, 8, 15, 16 wrap the side edges one hop at a time.

Totals: cost 40 · steps 20 — 12 rail ions batch 2 hops each; the 4 corner ions cost 8 hops each and set the clock.

24-Ancilla Ring-Rotation Schedule

Runs the 24-ancilla, 24-junction schedule from beginning to end. Each stabilizer keeps one fixed ancilla/dock for all six contacts within its wave.

Restart | Prev step | Run full schedule | Next step

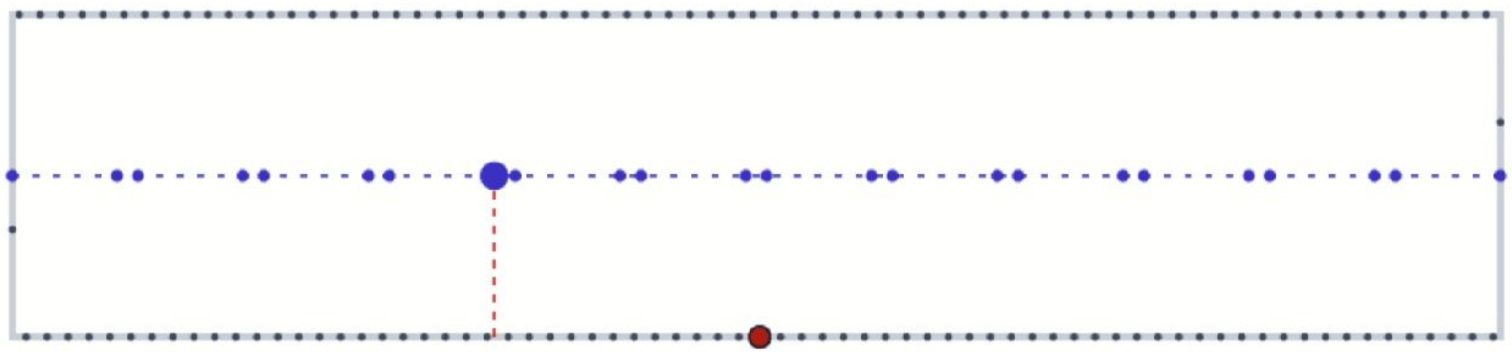
step	batch	contacts	rotate	phase	cost	done	avg used	final avg
1/8808	1/396	1	cw 13	rotate	1926	0/144	n/a	1213.3 steps

Step 1 of 8808: rotation 1/39: whole ring shifts toward shared offset

schedule step 1 / 8808; batch 1 / 396

rotation 1/39: whole ring shifts toward shared offset

rotate cw 13 | batch cost 1926 | steps 41



● spectator data ion ● X stabilizer data ● Z stabilizer data ● middle ancilla line ● active contact site

1 stabilizers measured together

Fixed ancilla wave: each stabilizer keeps the same dock across its six contacts.

Z | **Z_8_5** group0_ancilla20_dock120 -> data ion 108 | dock 120

Completed stabilizers

No stabilizer has finished all six contacts yet.

ROUTING SCHEME B

Highway Shuttling — Animated

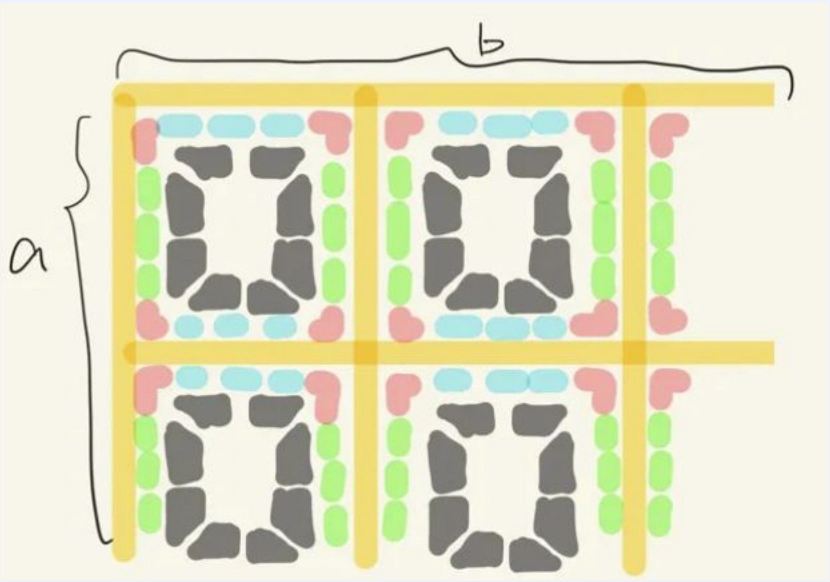
Schedule produced by routing_planner_B.py: spectators clear first, then movers thread the vacated segments — every step is a single, identical, collision-free hop batch.

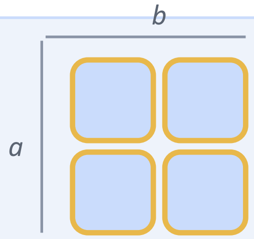
Totals: cost 52 · steps 46 — fully collision-free; ion 10 re-parks on the highway overhang to avoid a single-lane deadlock.

Unit Cell

Each of the N = a·b unit cells in the array carries 24 DC electrodes across three control classes.

ANATOMY OF ONE UNIT CELL





N = a·b cells

- Linear (horiz.) 6
- Linear (vert.) 6
- Junction 4, at corners
- Compensation 8, individual

12 linear + 4 junction + 8 compensation = 24 electrodes / cell

12

Linear control electrodes

- Broadcast horizontally & vertically, separately: **6 h + 6 v DACs.**
- Every DC is wired through a two-way switch.

4

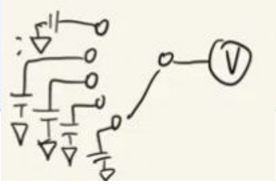
Junction control electrodes

- Broadcast control — **4 DACs** needed for the whole array.
- Every DC is wired through a switch.

8

Compensation electrodes

- Individually controlled — can't be broadcast.
- Uses a **1:100 demultiplexing network** to keep the DAC count down.



Broadcast Control Keeps the DAC Count Flat

Only the individually-tuned compensation electrodes scale with array size — and a 1:100 demux divides even that down.

$N = a \cdot b$ unit cells in the array — total trapping zones = $N - b$

12*2 = 24 DACs

Linear control — 6 broadcast horizontally, 6 vertically. Constant, independent of N.

4*2 = 8 DACs

Junction control — broadcast to every corner in the array. Constant, independent of N.

8N/100 DACs

Compensation — individually tuned per cell, but a 1:100 demultiplexing network divides the count by 100.

Total electrodes

$(12 + 4 + 8) \times N = 24N$

Total switches

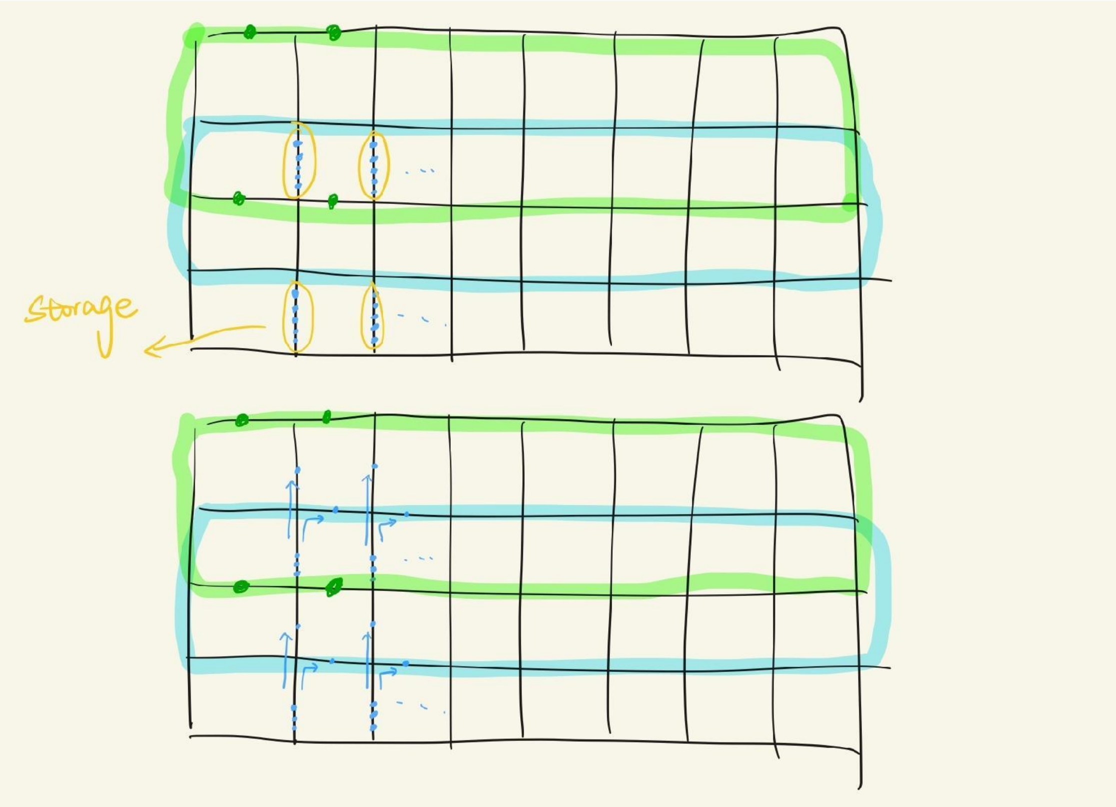
$24N \times 2$ (two-way) $= 48N$

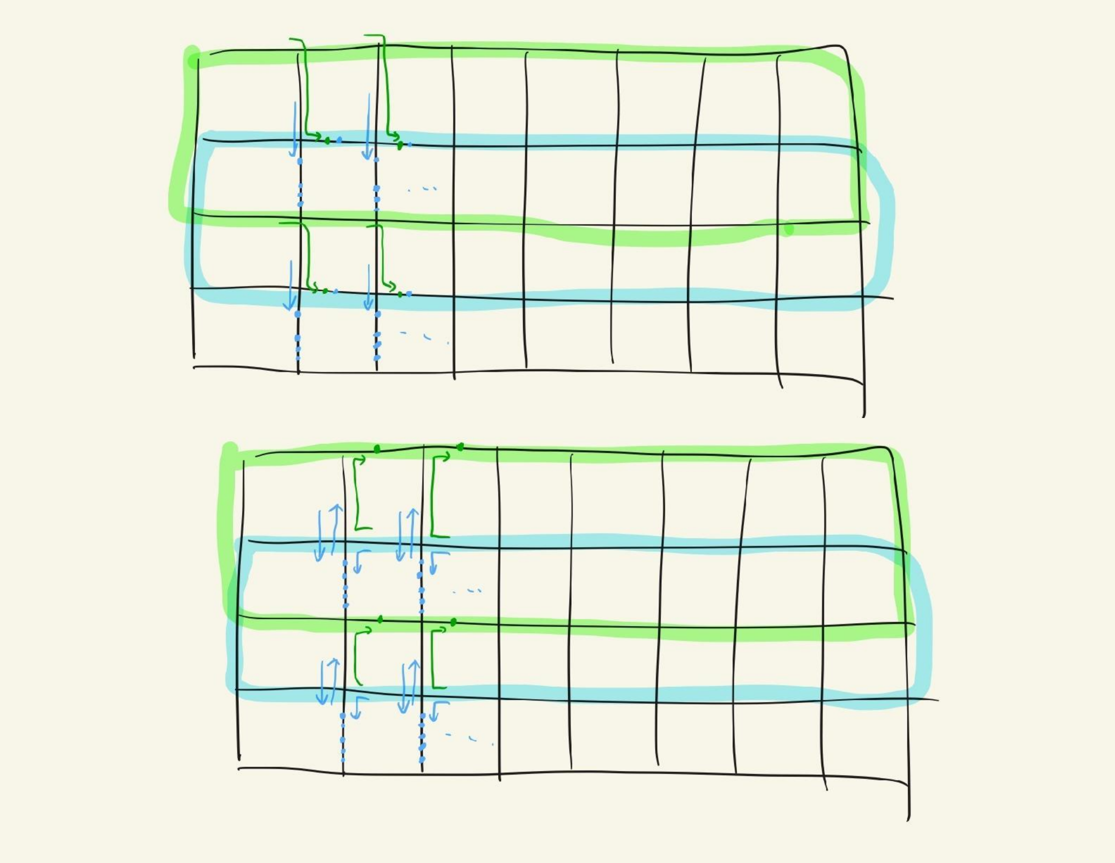
Total trapping zones

$N - b$

The scaling win: linear and junction control are broadcast, so their DAC count (12 and 4) never grows with the array. Only compensation scales with N — and the 1:100 demux network keeps even that manageable, so total DAC count stays far below the 24N raw electrode count.

Increase the number of ions per cell





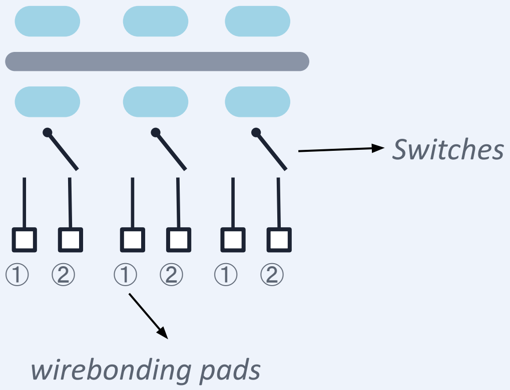
DC Electrode Pairs — How Many, and Who Switches Them

Two open questions on how each unit cell's DC electrodes are grouped and wired to the control switches.

1 How many pairs of DC are required for each unit cell?

3 / 4 / 5 Pairs?

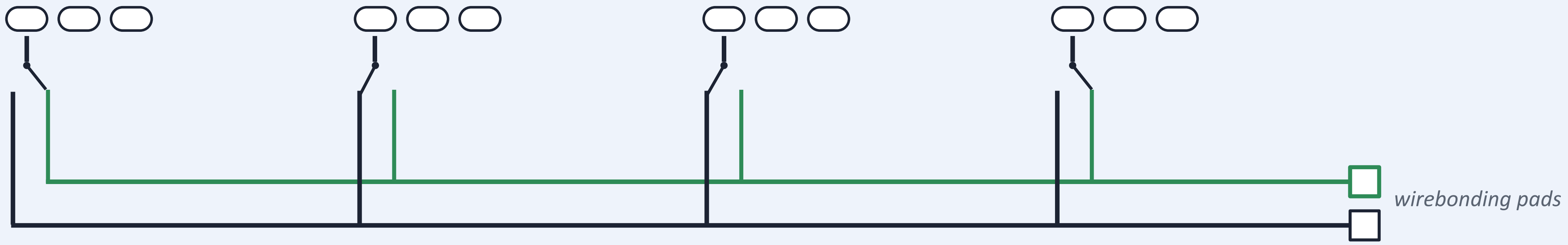
2 Every unit cell's DC needs to be controlled by a switch:



	Shuttle	Gate	..
Group ①	Move	Yes — stay	..
Group ②	Stay	No — shuttle for 100 µm	..

Shared Control Bus Across Unit Cells

Every unit cell's switch taps the same two lines — one shared bus per behavior, ending in a single wirebonding pad.



4 unit cells shown — pattern repeats across the array; every cell's switch taps the same shared green / dark control pair.

Question:

1. How to fabricate this type of switch? How scalable is it?
2. It’s not necessary, but is it possible to also integrate the **optical switches** on the PIC control?

Zone assignments

Different zones have different tasks, therefore different features

3 **Zone assignment:**

Unit cell contains: Gate operation lasers, SPAM lasers, Cooling lasers

Zones		Architecture
Logical qubit zones	Data qubit zones	Unit cell
	Ancilla qubit zones	Unit cell – gate lasers
T-gate factory		Unit cell
Bell state factory		Unit cell
Loading zones		Unit cell + PI laser – gate lasers